

Home > Posts >

Web3 Tip Jar: A Command-Line Tutorial with Foundry

August 14, 2026 · 25 min · 5202 words

► Table of Contents

This tutorial builds, deploys, and interacts with a simple `TipJar` smart contract entirely from the command line.

You will use:

- `forge` — compile, test, and deploy Solidity contracts
- `anvil` — run a local Ethereum blockchain
- `cast` — call contracts and send transactions

All three run inside a Podman container, so nothing is installed on the host.

No webpage or frontend is required.

The tutorial has two halves. The first uses `cast` against a bare local blockchain to see accounts, transactions, and blocks. The second deploys the `TipJar` contract and drives it from the shell.

1. Install Foundry in a Container

Foundry’s installer drops binaries into your home directory and edits your shell profile. Running it inside a container keeps all of that off the host, and makes the whole environment disposable — if anything goes wrong, delete the container and start again.

Everything below runs in Podman. Docker works identically; substitute `docker` for `podman` throughout.

First, a directory on the host to hold the project, so your work survives the container:

```
mkdir -p ~/tip-jar-work
```

Now start a long-lived Ubuntu container with that directory mounted:

```
podman run -dit \
  --name foundry \
  --volume ~/tip-jar-work:/work:Z \
  --workdir /work \
  ubuntu sleep infinity
```

Three flags worth understanding:

<code>-dit</code>	detached, but with a TTY so shells can attach later
<code>--name foundry</code>	a fixed name, so every later command can find it
<code>--volume ...:Z</code>	host directory → /work inside the container

`sleep infinity` is the container’s main process. A container lives exactly as long as its main process, so without it the container would start, run nothing, and immediately exit. Sleeping forever keeps it up so we can open shells into it.

The `:Z` suffix tells Podman to relabel the directory for SELinux. It is required on Fedora and RHEL, and harmless

everywhere else.

Now open a shell inside it:

```
podman exec -it foundry bash
```

The prompt changes to something like `root@a1b2c3d4:/work#`. You are now inside the container, and everything until §38 happens here.

The base Ubuntu image is deliberately bare, so install what Foundry needs:

```
apt-get update
apt-get install -y curl git ca-certificates
```

`git` is not optional — `forge init` uses it to fetch `forge-std`, the standard library the tests in §31 import.

Then the normal Foundry install:

```
curl -L https://foundry.paradigm.xyz | bash
source ~/.bashrc
foundryup
```

Verify:

```
forge --version
anvil --version
cast --version
```

If `forge` is not found after `foundryup`, the `source` did not take. Foundry installs to `~/.foundry/bin`, so this fixes it:

```
export PATH="$HOME/.foundry/bin:$PATH"
```

Getting Back In

The container keeps running in the background, so you can leave and return:

<code>exit</code>	leave the shell, container keeps running
<code>podman exec -it foundry bash</code>	open another shell in it
<code>podman ps</code>	confirm it is still up
<code>podman start foundry</code>	restart it after a reboot

Anything written to `/work` inside the container appears in `~/tip-jar-work` on the host, so you can edit `TipJar.sol` with your usual editor while running the commands in the container.

The one thing that does *not* survive is Foundry itself. It lives in the container's own filesystem, not in the mounted volume, so if you delete the container you will reinstall it. §39 covers cleanup and how to avoid that.

2. Create the Project

```
forge init tip-jar --empty
cd tip-jar
```

3. Create `src/TipJar.sol`

```

pragma solidity ^0.8.20;

contract TipJar {
    address public immutable owner;

    mapping(address => uint256) public tipsByAddress;
    uint256 public totalTips;

    event TipReceived(
        address indexed sender,
        uint256 amount,
        string message
    );

    event Withdrawn(address indexed owner, uint256 amount);

    constructor() {
        owner = msg.sender;
    }

    function tip(string calldata message) external payable {
        require(msg.value > 0, "Tip must be greater than zero");

        tipsByAddress[msg.sender] += msg.value;
        totalTips += msg.value;

        emit TipReceived(msg.sender, msg.value, message);
    }

    function getBalance() external view returns (uint256) {
        return address(this).balance;
    }

    function withdraw() external {
        require(msg.sender == owner, "Only owner can withdraw");

        uint256 amount = address(this).balance;
        require(amount > 0, "Nothing to withdraw");

        totalTips = 0;

        (bool success, ) = payable(owner).call{value: amount}("");
        require(success, "Withdrawal failed");

        emit Withdrawn(owner, amount);
    }
}

```

Two details worth noting before we run it:

- The contract has no `receive()` function, so ETH can only arrive through `tip()`. A plain ETH transfer to the contract address will revert. §32 adds one, and shows what can go wrong when it is added carelessly.
- `withdraw()` resets `totalTips` to zero, so `totalTips` tracks tips since the last withdrawal rather than the lifetime total. The per-address `tipsByAddress` mapping is never cleared.

4. Compile

```
forge build
```

The compiled artifact will appear under:

```
out/TipJar.sol/TipJar.json
```

5. Start Anvil

Open a second terminal on the host and get a second shell into the same container:

```
podman exec -it foundry bash
```

Then start the local blockchain:

```
anvil
```

Both shells are inside the same container, so Anvil's `127.0.0.1:8545` is reachable from the first one exactly as if everything were running natively. Nothing is published to the host, which means the chain is unreachable from outside the container. That is usually what you want. If you would rather point a host tool at it, recreate the container in §1 with `--publish 8545:8545` and start Anvil with `anvil --host 0.0.0.0`.

Leave it running for the rest of the tutorial. On startup it prints a screen like this, abridged here:

```
Available Accounts
=====

(0) 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266 (10000.000000000000000000 ETH)
(1) 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000.000000000000000000 ETH)
(2) ...
    ... 10 accounts in total ...

Private Keys
=====

(0) 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
(1) 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d
(2) ...
    ... 10 keys in total, one per account above ...

Wallet
=====
Mnemonic:      test test test test test test test test test test test junk
Derivation path: m/44'/60'/0'/0/

Chain ID
=====
31337

Base Fee
=====
1000000000

Gas Limit
=====
30000000

Genesis Timestamp
=====
(the Unix time at which you started Anvil, so this one varies)

Genesis Number
=====
0

Listening on 127.0.0.1:8545
```

Every value this tutorial needs comes off that screen. Five lines matter:

```
Available Accounts
(0) 0xf39Fd6...92266 → $OWNER      the tip jar owner
(1) 0x709979...dc79C8 → $TIPPER    sends the tips

Private Keys
(0) 0xac0974...f2ff80 → $OWNER_KEY signs transactions as the owner
(1) 0x59c699...78690d → $TIPPER_KEY signs transactions as the tipper

Listening on 127.0.0.1:8545
→ $RPC_URL    http://127.0.0.1:8545
```

Account (0) is just a convention here — any two of the ten accounts will do, as long as you keep each address paired with the private key printed at the same index.

Those values are identical on every machine, because Anvil derives them from the fixed mnemonic `test test ... junk` shown above. If you started Anvil with your own `--mnemonic`, copy the keys your own screen printed instead.

Here is what each section of the screen means:

Available Accounts

=====

10 pre-funded Ethereum accounts.

Format:

[(account address) (ETH balance)]

By default, Anvil gives each account 10,000 ETH.

Private Keys

=====

10 private keys, one for each account listed above.

A private key is used to sign transactions on behalf of its corresponding Ethereum address.

IMPORTANT: These Anvil private keys are for local development only. Never use them with real funds.

Wallet

=====

Mnemonic:

[seed phrase from which the accounts/private keys are deterministically generated]

Derivation Path:

[rule/path used to derive individual accounts from the mnemonic]

Example:

m/44'/60'/0'/0/0

m/44'/60'/0'/0/1

...

Chain ID

=====

Identifies the Ethereum-compatible network.

Every Ethereum network has a chain ID. It helps prevent a transaction signed for one network from being replayed on another network.

Example:

Anvil default Chain ID = 31337

Base Fee

=====

1000000000 wei

The minimum gas price per unit of gas for inclusion in a block under EIP-1559.

1000000000 wei = 1 gwei

NOTE:

The base fee is a price per unit of gas, not the total transaction fee.

Gas Limit

=====

The maximum amount of gas that can be consumed by a block.

This is NOT a gas fee.

For an individual transaction:

gas used × gas price = transaction fee

```
Genesis Timestamp
=====
```

The timestamp assigned to the genesis block.

The genesis block is block 0: the first block in the blockchain.

```
Genesis Number
=====
```

The block number at which the local blockchain starts.

```
Normally:
Genesis Number = 0
```

The `Listening on` line is the address of the JSON-RPC server, which every `cast` and `forge` command in this tutorial talks to.

6. Set Environment Variables

Back in your first container shell. Every command from here on refers to accounts through shell variables, never through a pasted address, so this is the only place you copy anything.

Start with the RPC endpoint, taken from Anvil's `Listening on` line:

```
export RPC_URL=http://127.0.0.1:8545
```

Now the two private keys, copied from the `Private Keys` section of the Anvil screen — index `(0)` for the owner, index `(1)` for the tipper:

```
export OWNER_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
export TIPPER_KEY=0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d
```

These are Anvil's public development keys, and they are only safe because the chain is local and the ETH is fake. Never use them for real funds.

You do not need to copy the addresses. An Ethereum address is derived from its private key, so `cast` can work them out:

```
export OWNER=$(cast wallet address --private-key "$OWNER_KEY")
export TIPPER=$(cast wallet address --private-key "$TIPPER_KEY")
```

Print them:

```
echo "RPC:      $RPC_URL"
echo "Owner:    $OWNER"
echo "Tipper:   $TIPPER"
```

Expected:

```
RPC:      http://127.0.0.1:8545
Owner:    0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266
Tipper:   0x70997970C51812dc3A010C7d01b50e0d17dc79C8
```

Those two addresses should be character-for-character the accounts printed at `(0)` and `(1)` under `Available Accounts` in the Anvil terminal. If they are not, the keys were copied from the wrong index.

One variable is still missing: `TIPJAR`, the contract's address. It cannot be set yet, because the contract does not exist until §16 deploys it.

7. Check the Current Block

```
cast block-number \
  --rpc-url "$RPC_URL "
```

You should see:

```
0
```

Anvil starts with the **genesis block**, block `0`.

8. Check the Account Balances

```
cast balance \
  "$OWNER" \
  --ether \
  --rpc-url "$RPC_URL "
```

You should see:

```
10000.000000000000000000
```

Anvil gave this account **10,000 test ETH** when it created the local blockchain. It is the same figure printed in parentheses after account `(0)` on the Anvil screen, and confirms that `$OWNER` really is one of the funded accounts.

Check the tipper:

```
cast balance \
  "$TIPPER" \
  --ether \
  --rpc-url "$RPC_URL "
```

It should also have:

```
10000.000000000000000000
```

9. Send 1 ETH From the Owner to the Tipper

```
cast send \
  "$TIPPER" \
  --value 1ether \
  --private-key "$OWNER_KEY" \
  --rpc-url "$RPC_URL "
```

`cast` uses the private key to sign a transaction saying:

```
Owner  — 1 ETH —>  Tipper
```

Anvil receives the transaction through its RPC server and mines it into a block.

The command prints a transaction receipt containing information such as:

```
blockNumber
from
to
gasUsed
status
transactionHash
```

10. Check the Block Number Again

```
cast block-number \
  --rpc-url "$RPC_URL"
```

You should now see:

```
1
```

The transaction caused Anvil to mine a new block:

```
Block 0      Genesis
  |
  ▼
Block 1      1 ETH transfer
```

11. Check the Recipient's Balance

```
cast balance \
  "$TIPPER" \
  --ether \
  --rpc-url "$RPC_URL"
```

You should now see:

```
10001.000000000000000000
```

The tipper received 1 ETH.

12. Check the Sender's Balance

```
cast balance \
  "$OWNER" \
  --ether \
  --rpc-url "$RPC_URL"
```

You will see slightly less than:

```
9999 ETH
```

rather than exactly 9999 ETH, because the owner paid:

```
1 ETH
+
transaction gas
```

The gas is also paid using Anvil's fake ETH.

13. Look at the Latest Block

```
cast block latest \  
  --rpc-url "$RPC_URL"
```

This shows information about the block Anvil just created, including:

```
number  
timestamp  
gasLimit  
gasUsed  
baseFeePerGas  
transactions
```

You can also ask for a specific block:

```
cast block 1 \  
  --rpc-url "$RPC_URL"
```

14. View a Balance in Wei

Try:

```
cast balance \  
  "$TIPPER" \  
  --rpc-url "$RPC_URL"
```

Without `--ether`, the balance is displayed in **wei**:

```
100010000000000000000000
```

because:

```
1 ETH = 1,000,000,000,000,000,000 wei  
      = 1018 wei
```

The `--ether` option simply tells `cast` to display the value in ETH instead of wei.

15. What Just Happened?

You interacted with Anvil in essentially the same way software interacts with a real Ethereum node:

```
cast  
|  
| JSON-RPC  
▼  
127.0.0.1:8545  
|  
▼
```

```
┌ Anvil  
│  
│ Block 0: Genesis  
│ Block 1: Transfer  
│  
│ Owner  
│   ~9999 ETH  
│  
│ Tipper  
│   10001 ETH  
└
```

The important commands so far:

```
# What block are we on?
cast block-number --rpc-url "$RPC_URL"

# How much ETH does an address have?
cast balance "$OWNER" --ether --rpc-url "$RPC_URL"

# Send a transaction
cast send "$TIPPER" --value 1ether \
  --private-key "$OWNER_KEY" --rpc-url "$RPC_URL"

# Inspect the latest block
cast block latest --rpc-url "$RPC_URL"
```

These commands demonstrate the basic Ethereum model:

```
accounts
  ↓
signed transactions
  ↓
Ethereum node
  ↓
blocks
  ↓
updated blockchain state
```

The next step is to deploy the `TipJar` contract with `forge` and then use `cast` to read from and write to it.

16. Deploy the Contract

Sign the deployment with `$OWNER_KEY`. Whoever deploys the contract runs its constructor, and the constructor sets `owner = msg.sender`, so this is what makes `$OWNER` the tip jar's owner:

```
forge create src/TipJar.sol:TipJar \
  --private-key "$OWNER_KEY" \
  --rpc-url "$RPC_URL" \
  --broadcast
```

You should see something like:

```
Deployer:      0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266    → matches $OWNER
Deployed to:    0x...                                           → $TIPJAR
Transaction hash: 0x...
```

`Deployer` is the address that `$OWNER_KEY` signed with, so it should match `$OWNER`. The address on the `Deployed to` line is the last variable you need:

```
export TIPJAR=0xPASTE_THE_DEPLOYED_TO_ADDRESS_HERE
```

Check it:

```
echo "$TIPJAR"
```

Unlike the account addresses, this one is not fixed. A contract address is computed from the deploying account and how many transactions it has already sent, so it depends on what you did earlier and changes every time you restart Anvil. Always copy the address your own `forge create` printed.

17. Verify the Contract Exists

```
cast code "$TIPJAR" \  
  --rpc-url "$RPC_URL"
```

A deployed contract returns bytecode beginning with:

```
0x...
```

If the result is only:

```
0x
```

there is no contract at that address, which usually means `$TIPJAR` was mistyped or is left over from an earlier Anvil run.

18. Read the Owner

```
cast call "$TIPJAR" \  
  "owner()(address)" \  
  --rpc-url "$RPC_URL"
```

This should match:

```
echo "$OWNER"
```

19. Check the Initial Balance

```
cast balance "$TIPJAR" \  
  --ether \  
  --rpc-url "$RPC_URL"
```

Initially:

```
0.000000000000000000
```

You can also use the contract function:

```
cast call "$TIPJAR" \  
  "getBalance()(uint256)" \  
  --rpc-url "$RPC_URL"
```

20. Send a Tip

Sign with `$TIPPER_KEY`, so the tip comes from `$TIPPER`:

```
cast send "$TIPJAR" \  
  "tip(string)" "Great work!" \  
  --value 0.1ether \  
  --private-key "$TIPPER_KEY" \  
  --rpc-url "$RPC_URL"
```

This sends `0.1 ETH` to the contract and stores the message.

21. Check the Contract Balance

```
cast balance "$TIPJAR" \
--ether \
--rpc-url "$RPC_URL"
```

Expected:

```
0.100000000000000000
```

22. Call `getBalance()`

```
cast call "$TIPJAR" \
  "getBalance()(uint256)" \
  --rpc-url "$RPC_URL"
```

For `0.1 ETH` the result is:

```
1000000000000000000
```

That value is in wei. Convert it to ETH:

```
cast from-wei 1000000000000000000
```

23. Check How Much the Tipper Has Tipped

```
cast call "$TIPJAR" \
  "tipsByAddress(address)(uint256)" "$TIPPER" \
  --rpc-url "$RPC_URL"
```

Expected:

```
1000000000000000000
```

Or, converted:

```
TIP_AMOUNT=$(cast call "$TIPJAR" \
  "tipsByAddress(address)(uint256)" "$TIPPER" \
  --rpc-url "$RPC_URL")

cast from-wei "$TIP_AMOUNT"
```

Expected:

```
0.100000000000000000
```

24. Check Total Tips

```
cast call "$TIPJAR" \
  "totalTips()(uint256)" \
  --rpc-url "$RPC_URL"
```

Expected after one `0.1 ETH` tip:

```
1000000000000000000
```

25. Send Another Tip

```
cast send "$TIPJAR" \  
  "tip(string)" "Second tip" \  
  --value 0.25ether \  
  --private-key "$TIPPER_KEY" \  
  --rpc-url "$RPC_URL"
```

Check the contract:

```
cast balance "$TIPJAR" \  
  --ether \  
  --rpc-url "$RPC_URL"
```

Expected:

```
0.350000000000000000
```

26. Test the Owner Restriction

Try withdrawing with `$TIPPER_KEY`:

```
cast send "$TIPJAR" \  
  "withdraw()" \  
  --private-key "$TIPPER_KEY" \  
  --rpc-url "$RPC_URL"
```

This should fail. `$TIPPER` is not the address stored in the contract's `owner` field, and the Solidity check is:

```
require(msg.sender == owner, "Only owner can withdraw");
```

27. Withdraw as the Owner

Sign with `$OWNER_KEY`, the key that deployed the contract:

```
cast send "$TIPJAR" \  
  "withdraw()" \  
  --private-key "$OWNER_KEY" \  
  --rpc-url "$RPC_URL"
```

This transfers the contract's ETH to the owner.

28. Confirm the Contract Is Empty

```
cast balance "$TIPJAR" \  
  --ether \  
  --rpc-url "$RPC_URL"
```

Expected:

```
0.000000000000000000
```

Check `totalTips`:

```
cast call "$TIPJAR" \
  "totalTips()(uint256)" \
  --rpc-url "$RPC_URL"
```

Expected:

```
0
```

The `tipsByAddress` mapping is not cleared, so the contract still records how much each address has tipped historically:

```
cast call "$TIPJAR" \
  "tipsByAddress(address)(uint256)" "$TIPPER" \
  --rpc-url "$RPC_URL"
```

Expected:

```
3500000000000000000
```

29. Inspect a Transaction

Every `cast send` prints a receipt containing a `transactionHash` field, and `forge create` prints the same value on its `Transaction hash` line. Put one in a variable:

```
export TX=0xPASTE_A_TRANSACTION_HASH_HERE
```

Then look it up. `cast tx` shows the transaction as it was submitted, that is, what the sender asked for:

```
cast tx "$TX" \
  --rpc-url "$RPC_URL"
```

`cast receipt` shows what happened when it was mined, including `status`, `gasUsed`, the block it landed in, and any events the contract emitted:

```
cast receipt "$TX" \
  --rpc-url "$RPC_URL"
```

The receipt for one of the `tip()` calls is where you can see the `TipReceived` event in the `logs` field.

If you would rather not copy hashes by hand, `cast send` can print JSON, so the hash can be captured straight into a variable:

```
export TX=$(cast send "$TIPJAR" \
  "tip(string)" "Captured hash" \
  --value 0.01ether \
  --private-key "$TIPPER_KEY" \
  --rpc-url "$RPC_URL" \
  --json | jq -r .transactionHash)

echo "$TX"
```

That needs `jq` installed. It also sends a real, if small, tip, so the contract balance is no longer zero after running it. Withdraw again if you want to get back to the state at the end of §28.

30. Inspect the Blockchain

Current block number:

```
cast block-number \  
  --rpc-url "$RPC_URL"
```

Latest block:

```
cast block latest \  
  --rpc-url "$RPC_URL"
```

31. Add Automated Tests

Driving the contract by hand is useful for learning, but tests are how you keep it working. Create `test/TipJar.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {Test} from "forge-std/Test.sol";
import {TipJar} from "../src/TipJar.sol";

contract TipJarTest is Test {
    TipJar private tipJar;

    address private owner = makeAddr("owner");
    address private tipper = makeAddr("tipper");

    function setUp() public {
        vm.prank(owner);
        tipJar = new TipJar();

        vm.deal(tipper, 1 ether);
    }

    function testOwnerIsDeployer() public view {
        assertEq(tipJar.owner(), owner);
    }

    function testTip() public {
        vm.prank(tipper);

        tipJar.tip{value: 0.25 ether}("Excellent!");

        assertEq(address(tipJar).balance, 0.25 ether);
        assertEq(tipJar.totalTips(), 0.25 ether);
        assertEq(tipJar.tipsByAddress(tipper), 0.25 ether);
    }

    function testOwnerCanWithdraw() public {
        vm.prank(tipper);
        tipJar.tip{value: 0.25 ether}("Excellent!");

        uint256 ownerBalanceBefore = owner.balance;

        vm.prank(owner);
        tipJar.withdraw();

        assertEq(address(tipJar).balance, 0);
        assertEq(tipJar.totalTips(), 0);
        assertEq(owner.balance, ownerBalanceBefore + 0.25 ether);
    }

    function testNonOwnerCannotWithdraw() public {
        vm.prank(tipper);
        tipJar.tip{value: 0.25 ether}("Excellent!");

        vm.prank(tipper);
        vm.expectRevert("Only owner can withdraw");

        tipJar.withdraw();
    }

    function testCannotSendZeroTip() public {
        vm.prank(tipper);
        vm.expectRevert("Tip must be greater than zero");

        tipJar.tip{value: 0}("No tip");
    }
}

```

The owner is a plain address created with `makeAddr`, and the contract is deployed through `vm.prank(owner)`. That matters: if the test contract itself were the owner, `withdraw()` would try to send ETH to a contract with no `receive()` function and revert.

Run:

```
forge test
```

More verbose:

```
forge test -vv
```

Full traces:

```
forge test -vvvv
```

32. Extension: Accept Plain ETH Transfers

Try sending ETH to the contract without naming a function:

```
cast send "$TIPJAR" \  
  --value 0.05ether \  
  --private-key "$TIPPER_KEY" \  
  --rpc-url "$RPC_URL"
```

It reverts. A transaction with no function signature carries empty calldata, and `TipJar` has nothing to run in that case. Contracts only accept a plain ETH transfer if they declare a `receive()` function, and only handle calls to unknown selectors if they declare a `fallback()`. `TipJar` has neither, so this also reverts:

```
cast send "$TIPJAR" \  
  "notAFunction()" \  
  --private-key "$TIPPER_KEY" \  
  --rpc-url "$RPC_URL"
```

Accepting plain transfers is worth having. It is what a wallet does when someone types the tip jar's address into a send field, without any idea that a `tip()` function exists.

The Naive Version

The obvious thing to write is an empty `receive()`:

```
receive() external payable {}
```

Rebuild, and redeploy, since a code change means a new contract:

```
forge build  
  
forge create src/TipJar.sol:TipJar \  
  --private-key "$OWNER_KEY" \  
  --rpc-url "$RPC_URL" \  
  --broadcast  
  
export TIPJAR=0xTHE_NEW_DEPLOYED_ADDRESS
```

Now the plain transfer succeeds:

```
cast send "$TIPJAR" \  
  --value 0.05ether \  
  --private-key "$TIPPER_KEY" \  
  --rpc-url "$RPC_URL"
```

But look at what the contract thinks happened:

```
cast balance "$TIPJAR" --rpc-url "$RPC_URL"

cast call "$TIPJAR" \
  "totalTips()(uint256)" \
  --rpc-url "$RPC_URL"

cast call "$TIPJAR" \
  "tipsByAddress(address)(uint256)" "$TIPPER" \
  --rpc-url "$RPC_URL"
```

```
5000000000000000000    the ETH is really there
0                        but totalTips never moved
0                        and nobody is credited for it
```

No ETH is lost — `withdraw()` sends `address(this).balance`, so the owner still gets it — but the contract’s own accounting now disagrees with its balance, and no event was emitted, so nothing downstream can even see the tip arrive.

The Fix

Route both entry points through one internal function, so there is a single place where a tip is recorded:

```
function tip(string calldata message) external payable {
    _tip(message);
}

receive() external payable {
    _tip("");
}

function _tip(string memory message) private {
    require(msg.value > 0, "Tip must be greater than zero");

    tipsByAddress[msg.sender] += msg.value;
    totalTips += msg.value;

    emit TipReceived(msg.sender, msg.value, message);
}
```

Three things to notice:

- `_tip` takes `string memory`, not `string calldata`. The `receive()` path has no calldata to point into, so the empty string it passes has to live in memory.
- A plain transfer has no room for a message, so it records an empty one. The `TipReceived` event still fires, which is what matters.
- `require(msg.value > 0, ...)` now guards the `receive()` path too, so a zero-value transfer to the contract reverts rather than silently doing nothing.

Rebuild, redeploy, and send another plain transfer. This time the accounting follows the ETH:

```
cast call "$TIPJAR" \
  "totalTips()(uint256)" \
  --rpc-url "$RPC_URL"
```

```
5000000000000000000
```

And the tipper is credited, exactly as if they had called `tip()`:

```
cast call "$TIPJAR" \
  "tipsByAddress(address)(uint256)" "$TIPPER" \
  --rpc-url "$RPC_URL"
```

The tests from §31 need no changes, since `tip()` behaves exactly as before:

```
forge test
```

33. Format and Rebuild

```
forge fmt
forge build
forge test
```

This is a good normal development loop.

34. Useful Commands

Compile

```
forge build
```

Test

```
forge test
```

Run one test contract only

```
forge test --match-contract TipJarTest -vv
```

Start local Ethereum

```
anvil
```

Deploy

```
forge create src/TipJar.sol:TipJar \
  --private-key "$OWNER_KEY" \
  --rpc-url "$RPC_URL" \
  --broadcast
```

Read contract state

```
cast call "$TIPJAR" \
  "owner()(address)" \
  --rpc-url "$RPC_URL"
```

Send a transaction

```
cast send "$TIPJAR" \
  "tip(string)" "hello" \
  --value 0.1ether \
  --private-key "$TIPPER_KEY" \
  --rpc-url "$RPC_URL"
```

Check ETH balance

```
cast balance "$TIPJAR" \
  --ether \
  --rpc-url "$RPC_URL"
```

ETH to wei

```
cast to-wei 0.1 ether
```

Wei to ETH

```
cast from-wei 10000000000000000
```

Derive an address from a private key

```
cast wallet address \  
  --private-key "$OWNER_KEY"
```

Inspect contract bytecode

```
cast code "$TIPJAR" \  
  --rpc-url "$RPC_URL"
```

35. Minimal Complete Session

Terminal 1:

```
podman start foundry  
podman exec -it foundry bash  
  
anvil
```

Terminal 2:

```
podman exec -it foundry bash  
  
# From Anvil's "Listening on" line  
export RPC_URL=http://127.0.0.1:8545  
  
# From Anvil's "Private Keys" section, indexes (0) and (1)  
export OWNER_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80  
export TIPPER_KEY=0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d  
  
# Derived from the keys; should match "Available Accounts" (0) and (1)  
export OWNER=$(cast wallet address --private-key "$OWNER_KEY")  
export TIPPER=$(cast wallet address --private-key "$TIPPER_KEY")  
  
forge build  
forge test  
  
forge create src/TipJar.sol:TipJar \  
  --private-key "$OWNER_KEY" \  
  --rpc-url "$RPC_URL" \  
  --broadcast
```

Copy the address from the `Deployed to` line that `forge create` just printed:

```
export TIPJAR=0xYOUR_DEPLOYED_ADDRESS
```

Send a tip:

```
cast send "$TIPJAR" \  
  "tip(string)" "Great work!" \  
  --value 0.1ether \  
  --private-key "$TIPPER_KEY" \  
  --rpc-url "$RPC_URL"
```

Check the balance:

```
cast balance "$TIPJAR" --ether --rpc-url "$RPC_URL"
```

Withdraw:

```
cast send "$TIPJAR" \
  "withdraw()" \
  --private-key "$OWNER_KEY" \
  --rpc-url "$RPC_URL"
```

Check the final balance:

```
cast balance "$TIPJAR" --ether --rpc-url "$RPC_URL"
```

36. Mental Model

The command-line workflow is:

```
TipJar.sol
  ↓
forge build
  ↓
forge test
  ↓
anvil
  ↓
forge create
  ↓
deployed TipJar
  ↓
cast call / cast send
```

The three tools have distinct roles:

```
forge = build, test, deploy
anvil = local Ethereum blockchain
cast  = interact with Ethereum
```

37. Important Anvil Behavior

Anvil is an ephemeral local blockchain. If you stop it and start it again:

```
anvil
```

your old local blockchain is gone. That means previous:

- contract deployments
- transactions
- contract balances
- contract addresses

no longer exist.

The new screen prints the same accounts and private keys as the old one, because Anvil derives them from the same default mnemonic every time. So `RPC_URL`, `OWNER_KEY`, `TIPPER_KEY`, `OWNER`, and `TIPPER` all stay valid. Only `TIPJAR` goes stale, along with any transaction hash you saved in `TX`:

```
forge create src/TipJar.sol:TipJar \
  --private-key "$OWNER_KEY" \
  --rpc-url "$RPC_URL" \
  --broadcast
```

```
export TIPJAR=0xNEW_DEPLOYED_ADDRESS
```

A `cast` command that returns `0x` or an empty result right after a restart is usually a `TIPJAR` still pointing at the contract on the old chain.

Stopping the container has the same effect, and more besides. Anvil keeps its chain in memory, so `podman stop foundry` ends the process and takes the chain with it. Restarting gives you a fresh chain at block `0`, exactly as if you had pressed `Ctrl-C` on Anvil itself.

38. Pointing `cast` at the Real Ethereum

Everything so far has run against Anvil. Nothing about `cast` is local, though — it is a JSON-RPC client, and Ethereum mainnet speaks the same JSON-RPC as the node on your laptop. Every command in this tutorial that only *reads* works against the real chain with one variable changed.

Reading is free. There is no key, no signature, no transaction, and nothing to lose. The node computes an answer and hands it back. So this whole section is safe to run, and none of it costs anything.

Open a third shell into the container, so you do not clobber the Anvil setup, and point it at mainnet:

```
podman exec -it foundry bash

export ETH_RPC_URL=https://ethereum-rpc.publicnode.com
```

The container reaches the internet through the host's network by default, so no extra Podman configuration is needed to talk to a real node.

Note the variable name. It is `ETH_RPC_URL`, not the `RPC_URL` the rest of this tutorial uses. `cast` reads `ETH_RPC_URL` from the environment on its own, so every command below can drop the `--rpc-url` flag that earlier sections pass explicitly.

That difference matters more than it looks. With no `--rpc-url` and no `ETH_RPC_URL`, `cast` quietly falls back to `http://localhost:8545`, which is Anvil. Everything in this section would then run against your local chain and report that mainnet's contracts do not exist.

So check where you actually are before going further:

```
cast chain
cast chain-id
cast block-number
cast client
```

`cast chain-id` must print `1`. If it prints `31337`, the export did not take and you are still talking to Anvil. Nothing below will work until that reads `1`.

Public endpoints are rate limited and occasionally down. If one stops answering, try another, or get a free key from a provider such as Alchemy, Infura, or QuickNode. The URL is the only thing that changes.

What Those Four Commands Told You

The chain ID is the number folded into every signature under EIP-155. It is the same field Anvil printed as `31337` on its startup screen, and the reason a transaction signed for one network cannot be replayed on another.

`cast client` reports which node software answered — `geth`, `nethermind`, `reth`, and so on. There is no single Ethereum program; there are several independent implementations that must agree, and you are talking to one of them.

The Chain Has a Beginning

Same command as §7, pointed somewhere older:

```
cast block 1
```

Block 1 of Ethereum mainnet was mined on 30 July 2015. `cast age` puts a date on any block number:

```
cast age 1
cast age latest
```

Compare the header fields against the ones Anvil produced in §13. The structure is identical. There are simply rather more blocks.

What Gas Actually Costs

```
cast gas-price
cast base-fee
```

Both come back in wei, which is not a useful unit for reading:

```
cast from-wei $(cast gas-price) gwei
```

Gwei is the customary unit for gas prices — 10^9 wei. Anvil hard-codes a base fee of 1 gwei, as its startup screen showed. Mainnet’s floats with demand, and watching it move over a few minutes is the cheapest possible introduction to Ethereum’s fee market.

Look Up an Account

`cast` resolves ENS names, so you rarely need to paste hex:

```
cast balance vitalik.eth --ether
cast resolve-name vitalik.eth
cast nonce vitalik.eth
```

The nonce is the number of transactions that address has ever sent. It is also what makes contract addresses predictable, which is why §16 noted that `TIPJAR` depends on how many transactions the deployer had already made.

There is a reverse command as well, going from an address back to a name:

The Best of Both

Anvil can start from a copy of the real chain:

```
anvil --fork-url https://ethereum-rpc.publicnode.com
```

Now `http://127.0.0.1:8545` has every mainnet contract, balance, and storage slot in it, but with instant blocks, the same ten pre-funded accounts, and money that is not real. You can call USDC, deploy `TipJar` alongside it, and break whatever you like. Nothing leaves your machine, and `Ctrl-C` puts it all back.

39. Container Lifecycle and Cleanup

Three verbs, and the difference between them matters:

```
podman stop foundry    pause it; filesystem intact, resumes with `start`
podman start foundry   bring it back, then `podman exec -it foundry bash`
podman rm -f foundry   delete it; the container filesystem is gone
```

Stopping is cheap and safe. Do it whenever you are done for the day:

```
podman stop foundry
```

Removing is the destructive one. What it takes with it:

GONE

Foundry install (~/.foundry)
apt packages (curl, git)
the container's shell history

KEPT

everything in ~/tip-jar-work
which is TipJar.sol, the tests,
foundry.toml, lib/, out/

Your work is on the host, so removing the container costs you the toolchain, not the project. Recreating it means repeating §1 from `podman run` onward.

Not Reinstalling Every Time

If you expect to delete and recreate the container, snapshot it once Foundry is installed:

```
podman commit foundry foundry-tipjar
```

That writes the container’s current filesystem out as a reusable image. New containers start from it with Foundry already present:

```
podman run -dit \
  --name foundry \
  --volume ~/tip-jar-work:/work:Z \
  --workdir /work \
  foundry-tipjar sleep infinity
```

`podman commit` captures whatever state the container happened to be in, which is convenient but not reproducible. For something you can rebuild from scratch, write a `Containerfile` on the host instead:

```
FROM ubuntu:24.04

RUN apt-get update && \
  apt-get install -y curl git ca-certificates && \
  rm -rf /var/lib/apt/lists/*

RUN curl -L https://foundry.paradigm.xyz | bash && \
  /root/.foundry/bin/foundryup

ENV PATH="/root/.foundry/bin:${PATH}"

WORKDIR /work
```

Build and use it the same way:

```
podman build -t foundry-tipjar .

podman run -dit \
  --name foundry \
  --volume ~/tip-jar-work:/work:Z \
  --workdir /work \
  foundry-tipjar sleep infinity
```

The `ENV PATH` line is what makes `forge` work without sourcing anything, which is why the Containerfile version never hits the “command not found” problem mentioned in §1.

Removing Everything

To leave no trace beyond your project files:

```
podman rm -f foundry
podman rmi foundry-tipjar ubuntu
```

`~/tip-jar-work` is untouched by all of this. Delete it yourself if you want the project gone too.

Security Note

Using Anvil's default private keys directly in terminal commands is fine for local development, because the keys are public and the ETH is fake.

For real networks, never expose real private keys in:

- shell history
- source code
- Git repositories
- frontend JavaScript
- committed `.env` files
- documentation

Use an encrypted keystore, hardware wallet, or another secure signing method instead.

The container helps a little here, but do not overestimate it. It keeps the Foundry install and this tutorial's throwaway keys off the host, and a bind mount means the container can only see `~/tip-jar-work` rather than your whole home directory. It does nothing about a real key you paste into a command — that key still ends up in the container's shell history, and the container still has unrestricted outbound network access.

Summary

Anvil provides the local blockchain:

```
Anvil
└─ http://127.0.0.1:8545
```

`cast` is a command-line client that talks to that blockchain:

```
cast —JSON-RPC→ Anvil
```

Using `cast`, you can:

- inspect account balances
- send ETH
- submit transactions
- inspect blocks
- call smart contracts
- read blockchain state

And `forge` compiles, tests, and deploys the contracts that `cast` talks to. That is the whole local development loop, with no frontend anywhere in sight.